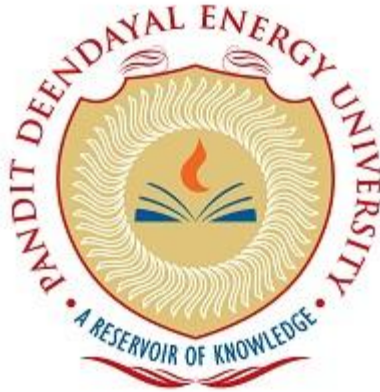




Pandit Deendayal Energy University (PDEU)
Department of Computer Science and Engineering,
School of Technology,
Design and Analysis of Algorithm
(24CS210P)

Name : Jay Patel

Roll No : 24BCP291

Lab Experiment 1

Objective:

Implementing the sorting algorithm: Bubble sort, Selection sort, Insertion sort, Merge sort and measuring the execution time and the number of steps of each.

Code:

```
#include <stdio.h>
#include <time.h>
long long steps = 0;
void merge(int a[], int l, int m, int r)
{
    int i, j, k;
    steps++;
    int n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++)
    {
        L[i] = a[l + i];
        steps++;
    }
    for (j = 0; j < n2; j++)
    {
        R[j] = a[m + 1 + j];
        steps++;
    }
    i = 0;
    j = 0;
    k = l;
    while (i < n1 && j < n2)
    {
        steps++;
        if (L[i] <= R[j])
            a[k++] = L[i++];
        else
            a[k++] = R[j++];
    }
    while (i < n1)
    {
```

```

        a[k++] = L[i++];
        steps++;
    }
    while (j < n2)
    {
        a[k++] = R[j++];
        steps++;
    }
}

```

```

void mergesort(int a[], int l, int r)
{
    steps++;
    if (l < r)
    {
        int m = (l + r) / 2;
        mergesort(a, l, m);
        mergesort(a, m + 1, r);
        merge(a, l, m, r);
    }
}

```

```

int partition(int arr[], int low, int high)
{
    int pivot = arr[high], i = low - 1;
    for (int j = low; j < high; j++)
    {
        steps++;
        if (arr[j] < pivot)
        {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
}

```

```

        steps++;
    }
}
int temp = arr[i + 1];
arr[i + 1] = arr[high];
arr[high] = temp;
return i + 1;
}

```

```

void quicksort(int arr[], int low, int high)
{
    steps++;
    if (low < high)
    {
        int pi = partition(arr, low, high);
        quicksort(arr, low, pi - 1);
        quicksort(arr, pi + 1, high);
    }
}

```

```

void copy(int original[], int duplicate[], int n)
{
    for (int i = 0; i < n; i++)
        duplicate[i] = original[i];
}

```

```

int main()
{
    clock_t start, end;
    double time_taken;
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int original[n];

```

```

printf("Enter elements:\n");
for (int i = 0; i < n; i++)
    scanf("%d", &original[i]);
int arr[n], ar[n], a[n], A[n], q[n];

copy(original, arr, n);
steps = 0;
start = clock();
for (int i = 0; i < n - 1; i++)
{
    int idx = i;
    for (int j = i + 1; j < n; j++)
    {
        steps++;
        if (arr[idx] > arr[j])
            idx = j;
    }
    if (idx != i)
    {
        int temp = arr[idx];
        arr[idx] = arr[i];
        arr[i] = temp;
        steps++;
    }
}
end = clock();
time_taken = (double)(end - start) / CLOCKS_PER_SEC;
printf("\nSelection Sort: ");
for (int i = 0; i < n; i++)
    printf("%d ", arr[i]);
printf("\nSteps=%lld Time=%f sec\n", steps, time_taken);

copy(original, ar, n);
steps = 0;

```

```

start = clock();
for (int i = 0; i < n; i++)
    for (int j = 0; j < n - 1 - i; j++)
    {
        steps++;
        if (ar[j] > ar[j + 1])
        {
            int temp = ar[j];
            ar[j] = ar[j + 1];
            ar[j + 1] = temp;
            steps++;
        }
    }
end = clock();
time_taken = (double)(end - start) / CLOCKS_PER_SEC;
printf("\nBubble Sort: ");
for (int i = 0; i < n; i++)
    printf("%d ", ar[i]);
printf("\nSteps=%lld Time=%f sec\n", steps, time_taken);

```

```

copy(original, a, n);
steps = 0;
start = clock();
for (int i = 1; i < n; i++)
{
    int key = a[i], j = i - 1;
    while (j >= 0 && a[j] > key)
    {
        a[j + 1] = a[j];
        j--;
        steps++;
    }
    a[j + 1] = key;
}

```

```
end = clock();
time_taken = (double)(end - start) / CLOCKS_PER_SEC;
printf("\nInsertion Sort: ");
for (int i = 0; i < n; i++)
    printf("%d ", a[i]);
printf("\nSteps=%lld Time=%f sec\n", steps, time_taken);
```

```
copy(original, A, n);
steps = 0;
start = clock();
mergesort(A, 0, n - 1);
end = clock();
time_taken = (double)(end - start) / CLOCKS_PER_SEC;
printf("\nMerge Sort: ");
for (int i = 0; i < n; i++)
    printf("%d ", A[i]);
printf("\nSteps=%lld Time=%f sec\n", steps, time_taken);
```

```
copy(original, q, n);
steps = 0;
start = clock();
quicksort(q, 0, n - 1);
end = clock();
time_taken = (double)(end - start) / CLOCKS_PER_SEC;
printf("\nQuick Sort: ");
for (int i = 0; i < n; i++)
    printf("%d ", q[i]);
printf("\nSteps=%lld Time=%f sec\n", steps, time_taken);
}
```

OUTPUT:

Enter number of elements: 6

Enter elements:

78

44

56

92

21

89

Selection Sort: 21 44 56 78 89 92

Steps=18 Time=0.000000 sec

Bubble Sort: 21 44 56 78 89 92

Steps=22 Time=0.000000 sec

Insertion Sort: 21 44 56 78 89 92

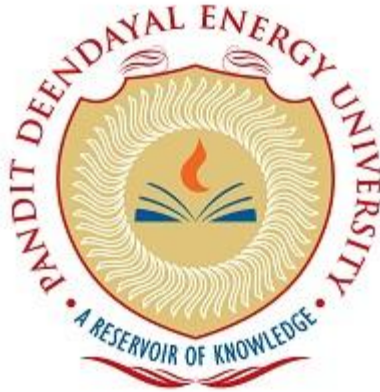
Steps=7 Time=0.000000 sec

Merge Sort: 21 44 56 78 89 92

Steps=48 Time=0.000000 sec

Quick Sort: 21 44 56 78 89 92

Steps=27 Time=0.000000 sec



Pandit Deendayal Energy University (PDEU)
Department of Computer Science and Engineering,
School of Technology,
Design and Analysis of Algorithm
(24CS210P)

Name : Jay Patel

Roll No : 24BCP291

Lab Experiment 2

Objective:

1. Implementing **searching algorithms**.
2. Measuring their **execution time** and **number of iterations** in best case, worst case and average case.

Code:

```
#include<stdio.h>
#include<time.h>
void binarysearch(int a[], int low, int high, int key, int *count)
{
    if (low > high)
    {
        printf("Element not found...\n");
        return;
    }

    int mid = low + (high - low) / 2;
    (*count)++;

    if (a[mid] == key)
    {
        printf("Element found...\n");
        return;
    }
    else if (a[mid] > key)
    {
        binarysearch(a, low, mid - 1, key, count);
    }
    else
    {
        binarysearch(a, mid + 1, high, key, count);
    }
}

int main()
{
    clock_t start,end;
    double cpu;
    printf("==== Choose searching method =====\n");
    printf("1. Linear search \n");
    printf("2. Binary search \n");
    int count =0 ;
    int choice;
    scanf("%d",&choice);
    int n;
    printf("Enter the array size : ");
    scanf("%d", &n);
    int a[n];
```

```

for(int i=0; i<n ;i++)
scanf("%d",&a[i]);
int find;
printf("Enter the searching element : ");
scanf("%d", &find);
start = clock();
if( choice == 1 )
{
    // linear search
    for( int i=0; i< n ; i++)
    {
        count++;
        if( a[i] == find )
        {
            printf("Element found on index %d\n",i);
            break;
        }
    }
    printf("Count = %d \n",count);
}
else
{
    // Binary Search
    for(int i=0; i<n; i++)
    { // sorting the array
        for(int j=0; j< n-1-i; j++)
        {
            if( a[j] > a[j+1] )
            {
                count++;
                int temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }
    if( a[0] == find )
    {
        printf("Element found ...\n");
        count++;
    }
    else if ( a[n-1] == find )
    {

```

```

        count++;
        printf("Element found...\n");
    }
    else{ binarysearch(a,0,n-1, find, &count); }
    printf("Count = %d\n",count);

}
end = clock(); cpu = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Execution time : %f seconds\n",cpu);
return 0;
}

```

OUTPUT:

```

===== Choose searching method =====
1. Linear search
2. Binary search
1
Enter the array size : 8
45
7
3
24
7
9
64
34
Enter the searching element : 64
Element found on index 6
Count = 7
Execution time : 0.001000 seconds

```



Pandit Deendayal Energy University (PDEU)
Department of Computer Science and Engineering,
School of Technology,
Design and Analysis of Algorithm
(24CS210P)

Name : Jay Patel
Roll No : 24BCP291

Lab Experiment 3

Objective:

Implementing arithmetic operations on integers of unlimited size using linked list.

Code:

```
#include <bits/stdc++.h>
using namespace std;
struct node
{
    int data;
    struct node *link;
};
struct node *createnode(struct node *head, int val)
{
    struct node *newn = new node;
    newn->data = val;
    newn->link = NULL;

    if (head == NULL)
    {
        head = newn;
    }
    else
    {
        struct node *temp = head;
        while (temp->link != NULL)
        {
            temp = temp->link;
        }
        temp->link = newn;
    }
    return head;
}
int main()
{
    int a, b;
    cout << "Enter number a = ";
    cin >> a;
    cout << "Enter number b = ";
    cin >> b;

    struct node *p = NULL;
    struct node *q = NULL;
    string p1 = to_string(a);
    string q1 = to_string(b);
```

```

for (int i = p1.length() - 1; i >= 0; i--)
{
    p = createnode(p, p1[i] - '0');
}
for (int i = q1.length() - 1; i >= 0; i--)
{
    q = createnode(q, q1[i] - '0');
}

```

```

struct node *temp = p;
long long sum = 0;
int ten = 1;
while (temp != NULL)
{
    sum = sum + temp->data * ten;
    ten = ten * 10;
    temp = temp->link;
}
ten = 1;
temp = q;
while (temp != NULL)
{
    sum = sum + temp->data * ten;
    ten = ten * 10;
    temp = temp->link;
}
cout << "Addition = " << sum << endl;

```

```

long long diff = 0;
ten = 1;
temp = p;
while (temp != NULL)
{
    diff = diff + temp->data * ten;
    ten = ten * 10;
    temp = temp->link;
}
ten = 1;
temp = q;
while (temp != NULL)
{
    diff = diff - temp->data * ten;
    ten = ten * 10;
}

```

```

        temp = temp->link;
    }
    cout << "Subtraction = " << diff << endl;

    long long num1 = 0, num2 = 0;
    ten = 1;
    temp = p;
    while (temp != NULL)
    {
        num1 = num1 + temp->data * ten;
        ten = ten * 10;
        temp = temp->link;
    }
    ten = 1;
    temp = q;
    while (temp != NULL)
    {
        num2 = num2 + temp->data * ten;
        ten = ten * 10;
        temp = temp->link;
    }
    long long mul = num1 * num2;
    cout << "Multiplication = " << mul << endl;

    long long expo = 1;
    for (int i = 0; i < num2; i++)
    {
        expo = expo * num1;
    }
    cout << "Exponential = " << expo << endl;
    return 0;
}

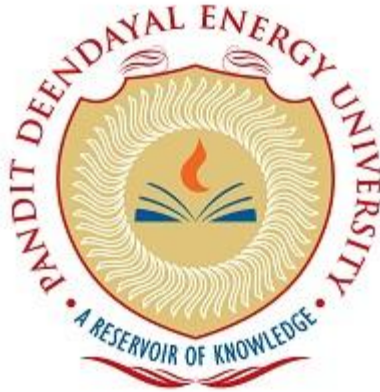
```

OUTPUT:

```

Enter number a = 3
Enter number b = 5
Addition = 8
Subtraction = -2
Multiplication = 15
Exponential = 243

```

Pandit Deendayal Energy University (PDEU)
Department of Computer Science and Engineering,
School of Technology,
Design and Analysis of Algorithm
(24CS210P)

Name : Jay Patel

Roll No : 24BCP291

Lab Experiment 4

Objective:

Implementing city database management using linked lists and array-based list. Comparing the execution time of both.

Code:

```
#include <bits/stdc++.h>
#include <time.h>
using namespace std;

struct node
{
    string city;
    int x;
    int y;
    struct node *link;
};

struct arrnode
{
    string city;
    int x;
    int y;
};

int main()
{
    clock_t start, end;
    double time_taken;

    int n;
    cout << "\nEnter number of cities = ";
    cin >> n;

    node *head = nullptr;

    start = clock();

    for (int i = 0; i < n; i++)
    {
        string s;
        int x, y;
        node *newn = new node();

        cout << "\nEnter city name = ";
        cin >> s;
        newn->city = s;
```

```

    cout << "\nEnter x coordinate = ";
    cin >> x;
    newn->x = x;

    cout << "\nEnter y coordinate = ";
    cin >> y;
    newn->y = y;

    newn->link = nullptr;

    if (head == nullptr)
        head = newn;
    else
    {
        node *temp = head;
        while (temp->link != nullptr)
            temp = temp->link;

        temp->link = newn;
    }
}

end = clock();

cout << "\nLinked List Insert Time = " << (double)(end - start) / CLOCKS_PER_SEC << "
sec";

string tar;

cout << "\n----- Searching a record -----" << endl;
cout << "Enter name of the city = ";
cin >> tar;

start = clock();

node *temp = head;
bool found = false;

while (temp != nullptr)
{
    if (temp->city == tar)
    {

```

```

        cout << "\nCity found Successfully...";
        cout << "\nCity Name = " << temp->city;
        cout << "\nCoordinates = " << temp->x << ", " << temp->y;
        found = true;
        break;
    }
    temp = temp->link;
}

end = clock();

cout << "\nLinked List Search Time = " << (double)(end - start) / CLOCKS_PER_SEC <<
" sec";

string del;

cout << "\n----- Deleting a record -----";
cout << "\nEnter name of the city = ";
cin >> del;

start = clock();

temp = head;
node *prev = nullptr;
found = false;

while (temp != nullptr)
{
    if (temp->city == del)
    {
        if (temp == head)
            head = temp->link;
        else
            prev->link = temp->link;

        delete temp;
        found = true;
        break;
    }

    prev = temp;
    temp = temp->link;
}

```

```

end = clock();

cout << "\nLinked List Delete Time = " << (double)(end - start) / CLOCKS_PER_SEC <<
" sec";

cout << "\n----- Printing records in range -----";

int x1, y1, x2, y2;

cout << "\nEnter x1 coordinate = ";
cin >> x1;
cout << "\nEnter y1 coordinate = ";
cin >> y1;
cout << "\nEnter x2 coordinate = ";
cin >> x2;
cout << "\nEnter y2 coordinate = ";
cin >> y2;

start = clock();

temp = head;

while (temp != nullptr)
{
    if (x1 <= temp->x && temp->x <= x2 && y1 <= temp->y && temp->y <= y2)
    {
        cout << "City name " << temp->city << endl;
        cout << "Coordinates = " << temp->x << ", " << temp->y << endl;
    }
    temp = temp->link;
}
end = clock();
cout << "\nLinked List Range Time = " << (double)(end - start) / CLOCKS_PER_SEC <<
" sec";
cout << "\n\n---- Array implementation ----\n";
arrnode arr[n];
start = clock();
for (int i = 0; i < n; i++)
{
    cout << "\nEnter city name = ";
    cin >> arr[i].city;
    cout << "\nEnter x coordinate = ";

```

```

        cin >> arr[i].x;
        cout << "\nEnter y coordinate = ";
        cin >> arr[i].y;
    }
    end = clock();
    cout << "\nArray Insert Time = " << (double)(end - start) / CLOCKS_PER_SEC << " sec";
    cout << "\nEnter city to search = ";
    cin >> tar;
    start = clock();
    for (int i = 0; i < n; i++)
    {
        if (arr[i].city == tar)
        {
            cout << "\nFound " << arr[i].city;
            break;
        }
    }
    end = clock();
    cout << "\nArray Search Time = " << (double)(end - start) / CLOCKS_PER_SEC << "
sec";
    cout << "\nEnter city to delete = ";
    cin >> del;
    start = clock();
    for (int i = 0; i < n; i++)
    {
        if (arr[i].city == del)
        {
            for (int j = i; j < n - 1; j++)
                arr[j] = arr[j + 1];

            n--;
            break;
        }
    }
    end = clock();
    cout << "\nArray Delete Time = " << (double)(end - start) / CLOCKS_PER_SEC << "
sec";
    cout << "\nEnter range again\n";
    cout << "\nEnter x1 coordinate = ";
    cin >> x1;
    cout << "\nEnter y1 coordinate = ";
    cin >> y1;
    cout << "\nEnter x2 coordinate = ";

```

```

cin >> x2;
cout << "\nEnter y2 coordinate = ";
cin >> y2;
start = clock();
for (int i = 0; i < n; i++)
{
    if (x1 <= arr[i].x && arr[i].x <= x2 && y1 <= arr[i].y && arr[i].y <= y2)
    {
        cout << arr[i].city << endl;
    }
}
end = clock();
cout << "\nArray Range Time = " << (double)(end - start) / CLOCKS_PER_SEC << "
sec";
}

```

OUTPUT:

```

Enter number of cities = 3

Enter city name = guj

Enter x coordinate = 2

Enter y coordinate = 3

Enter city name = ahmd

Enter x coordinate = 3

Enter y coordinate = 10

Enter city name = morbi

Enter x coordinate = 5

Enter y coordinate = 6

Linked List Insert Time = 23.011 sec

```

```
----- Searching a record -----  
Enter name of the city = morbi  
  
City found Successfully...  
City Name = morbi  
Coordinates = 5,6  
Linked List Search Time = 0 sec  
----- Deleting a record -----  
Enter name of the city = morbi  
  
Linked List Delete Time = 0 sec  
----- Printing records in range -----  
Enter x1 coordinate = 1  
  
Enter y1 coordinate = 1  
  
Enter x2 coordinate = 5  
  
Enter y2 coordinate = 5  
City name guj  
Coordinates = 2,3
```

```
Linked List Range Time = 0 sec  
  
---- Array implementation ----  
  
Enter city name = rajula  
  
Enter x coordinate = 3  
  
Enter y coordinate = 7  
  
Enter city name = amreli  
  
Enter x coordinate = 9  
  
Enter y coordinate = 9  
  
Enter city name = gandhinagar  
  
Enter x coordinate = 1  
  
Enter y coordinate = 2  
  
Array Insert Time = 35.88 sec
```


Enter city to search = rajula

Found rajula

Array Search Time = 0 sec

Enter city to delete = rajula

Array Delete Time = 0 sec

Enter range again

Enter x1 coordinate = 1

Enter y1 coordinate = 2

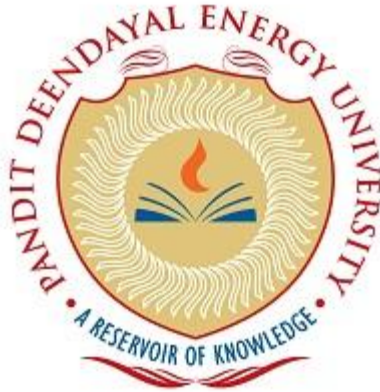
Enter x2 coordinate = 10

Enter y2 coordinate = 10

amreli

gandhinagar

Array Range Time = 0 sec



Pandit Deendayal Energy University (PDEU)
Department of Computer Science and Engineering,
School of Technology,
Design and Analysis of Algorithm
(24CS210P)

Name : Jay Patel

Roll No : 24BCP291

Lab Experiment 5

Objective:

Implement min, max and median using divide and conquer method. Perform the standard matrix multiplication and Strassen's algorithm for O^2 matrix multiplication.

Code 1 (mean, median, max):

```
#include <iostream>
using namespace std;
int mx, mn; int i=0;
void DivideandConquer(int a[], int low, int high)
{
    if(low < high)
    {
        int piv = a[low];
        int i = low + 1;
        int j = high;
        while(i <= j)
        {
            while(a[i] <= piv && i <= high)
                i++;

            while(a[j] > piv)
                j--;

            if(i < j)
                swap(a[i], a[j]);
        }
        swap(a[low], a[j]);
        DivideandConquer(a, low, j - 1);
        DivideandConquer(a, j + 1, high);
    }
}

void mxmin(int a[], int low, int high)
{
    if(low == high)
    {
        mx = mn = a[low];
        return;
    }
    if(low == high - 1)
    {
        if(a[low] > a[high])
        {
            mx = a[low];
            mn = a[high];
        }
        else
```

```

        {
            mx = a[high];
            mn = a[low];
        }
        return;
    }
    int mid = (low + high) / 2;
    int max1, min1, max2, min2;
    mxmin(a, low, mid);
    max1 = mx;
    min1 = mn;
    mxmin(a, mid + 1, high);
    max2 = mx;
    min2 = mn;
    if(max1 > max2)
        mx = max1;
    else
        mx = max2;
    i++;
    cout<<"Local Max "<<i<<" = "<<mx<<endl;
    if(min1 < min2)
        mn = min1;
    else
        mn = min2;
    cout<<"Local Min "<<i<<" = "<<mn<<endl;
    cout<<"\n";
}
int main()
{
    int n;
    cout << "Enter size: ";
    cin >> n;
    int a[n];
    cout << "Enter elements:\n";
    for(int i = 0; i < n; i++)
        cin >> a[i];
    mxmin(a, 0, n-1);
    cout << "Maximum = " << mx << endl;
    cout << "Minimum = " << mn << endl;

    cout<<"\n----- Median using Divide and Conquer -----"<<endl;
    DivideandConquer(a,0,n-1);
    if(n % 2 == 0)

```

```
    cout << "Median = " << (a[n/2] + a[n/2 - 1]) / 2.0;
    else
    cout << "Median = " << a[n/2];
    return 0;
}
```

OUTPUT:

```
Enter size: 7
Enter elements:
23
5
6
89
42
6
8
Local Max 1 = 89
Local Min 1 = 5

Local Max 2 = 42
Local Min 2 = 6

Local Max 3 = 89
Local Min 3 = 5

Maximum = 89
Minimum = 5

----- Median using Divide and Conquer -----
Median = 8
```

Code 2 (Standard Multiplication):

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    int n;
    cout << "Enter order of matrix = ";
    cin >> n;
    int mat1[n][n], mat2[n][n], res[n][n];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            res[i][j] = 0;
    cout << "----- Enter Matrix 1 -----\\n";
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> mat1[i][j];
    cout << "----- Enter Matrix 2 -----\\n";
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> mat2[i][j];

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            for (int k = 0; k < n; k++)
            {
                res[i][j] += mat1[i][k] * mat2[k][j];
            }
        }
    }
    cout << "----- Result Matrix -----\\n";
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
            cout << res[i][j] << " ";
        cout << "\\n";
    }
    return 0;
}
```

OUTPUT:

```
Enter order of matrix = 2
----- Enter Matrix 1 -----
1
4
6
3
----- Enter Matrix 2 -----
3
7
8
9
----- Result Matrix -----
35 43
42 69
```

Code 3 (Stressen's Algorithm):

```
#include <iostream>

using namespace std;

void add(int A[][10], int B[][10], int C[][10], int size)
{
    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            C[i][j] = A[i][j] + B[i][j];
}

void sub(int A[][10], int B[][10], int C[][10], int size)
{
    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            C[i][j] = A[i][j] - B[i][j];
}
```

```

}

void strassen(int A[][10], int B[][10], int C[][10], int n)
{
    if (n == 2)
    {
        int M1 = (A[0][0] + A[1][1]) * (B[0][0] + B[1][1]);
        int M2 = (A[1][0] + A[1][1]) * B[0][0];
        int M3 = A[0][0] * (B[0][1] - B[1][1]);
        int M4 = A[1][1] * (B[1][0] - B[0][0]);
        int M5 = (A[0][0] + A[0][1]) * B[1][1];
        int M6 = (A[1][0] - A[0][0]) * (B[0][0] + B[0][1]);
        int M7 = (A[0][1] - A[1][1]) * (B[1][0] + B[1][1]);

        C[0][0] = M1 + M4 - M5 + M7;
        C[0][1] = M3 + M5;
        C[1][0] = M2 + M4;
        C[1][1] = M1 - M2 + M3 + M6;
        return;
    }
}

```

```

int mid = n / 2;

```

```

int A11[10][10], A12[10][10], A21[10][10], A22[10][10];
int B11[10][10], B12[10][10], B21[10][10], B22[10][10];
int C11[10][10], C12[10][10], C21[10][10], C22[10][10];

```

```

int M1[10][10], M2[10][10], M3[10][10], M4[10][10];
int M5[10][10], M6[10][10], M7[10][10];

```



```
int T1[10][10], T2[10][10];
```

```
for (int i = 0; i < mid; i++)
```

```
    for (int j = 0; j < mid; j++)
```

```
    {
```

```
        A11[i][j] = A[i][j];
```

```
        A12[i][j] = A[i][j + mid];
```

```
        A21[i][j] = A[i + mid][j];
```

```
        A22[i][j] = A[i + mid][j + mid];
```

```
        B11[i][j] = B[i][j];
```

```
        B12[i][j] = B[i][j + mid];
```

```
        B21[i][j] = B[i + mid][j];
```

```
        B22[i][j] = B[i + mid][j + mid];
```

```
    }
```

```
add(A11, A22, T1, mid);
```

```
add(B11, B22, T2, mid);
```

```
strassen(T1, T2, M1, mid);
```

```
add(A21, A22, T1, mid);
```

```
strassen(T1, B11, M2, mid);
```

```
sub(B12, B22, T2, mid);
```

```
strassen(A11, T2, M3, mid);
```

```
sub(B21, B11, T2, mid);
```

```
strassen(A22, T2, M4, mid);
```

```
add(A11, A12, T1, mid);  
strassen(T1, B22, M5, mid);
```

```
sub(A21, A11, T1, mid);  
add(B11, B12, T2, mid);  
strassen(T1, T2, M6, mid);
```

```
sub(A12, A22, T1, mid);  
add(B21, B22, T2, mid);  
strassen(T1, T2, M7, mid);
```

```
add(M1, M4, T1, mid);  
sub(T1, M5, T2, mid);  
add(T2, M7, C11, mid);
```

```
add(M3, M5, C12, mid);  
add(M2, M4, C21, mid);
```

```
sub(M1, M2, T1, mid);  
add(T1, M3, T2, mid);  
add(T2, M6, C22, mid);
```

```
for (int i = 0; i < mid; i++)  
    for (int j = 0; j < mid; j++)  
    {  
        C[i][j] = C11[i][j];  
        C[i][j + mid] = C12[i][j];  
        C[i + mid][j] = C21[i][j];  
        C[i + mid][j + mid] = C22[i][j];  
    }
```

```

    }
}

int main()
{
    int n;
    cout << "Enter order (power of 2): ";
    cin >> n;
    int A[10][10], B[10][10], C[10][10];
    cout << "Enter Matrix A:\n";
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> A[i][j];
    cout << "Enter Matrix B:\n";
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> B[i][j];
    strassen(A, B, C, n);
    cout << "Result:\n";
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
            cout << C[i][j] << " ";
        cout << endl;
    }
}

```

OUTPUT:

Enter order (power of 2): 4

Enter Matrix A:

3

6

8

5

2

1

4

7

88

9

7

4

2

1

4

6

Enter Matrix B:

8

0

7

6

5

4

3

5

67

4

3

5

3

6

7

8

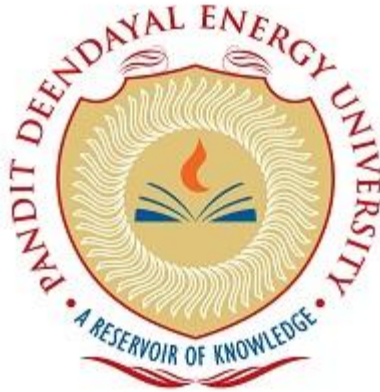
Result:

605 86 98 128

310 62 78 93

1230 88 692 640

307 56 71 85



Pandit Deendayal Energy University (PDEU)
Department of Computer Science and Engineering,
School of Technology,
Design and Analysis of Algorithm
(24CS210P)

Name : Jay Patel
Roll No : 24BCP291

Lab Experiment 6

Objective:

Implement interval scheduling algorithm for **n** events with their starting and ending times.

Code:

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    int n;
    cout << "Enter number of events = ";
    cin >> n;
    vector<vector<int>> a(n, vector<int>(3));
    cout << "Enter (start, end):\n";
    for (int i = 0; i < n; i++)
    {
        a[i][0] = 'A' + i;
        cout << char(a[i][0]) << " : ";
        cin >> a[i][1] >> a[i][2];
    }
    sort(a.begin(), a.end(), [](auto &x, auto &y)
        { return x[2] < y[2]; });
    cout << "\nSelected Events:\n";
    int last = a[0][2];
    cout << char(a[0][0]) << " (" << a[0][1] << ", " << a[0][2] << ")\n";
    for (int i = 1; i < n; i++)
    {
        if (a[i][1] >= last)
        {
            cout << char(a[i][0]) << " (" << a[i][1] << ", " << a[i][2] << ")\n";
            last = a[i][2];
        }
    }
    return 0;
}
```

OUTPUT:

Enter number of events = 6

Enter (start, end):

A : 1

3

B : 2

5

C : 4

6

D : 6

7

E : 5

9

F : 8

9

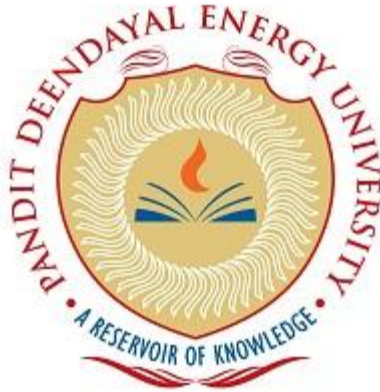
Selected Events:

A (1, 3)

C (4, 6)

D (6, 7)

F (8, 9)



Pandit Deendayal Energy University (PDEU)
Department of Computer Science and Engineering,
School of Technology,
Design and Analysis of Algorithm
(24CS210P)

Name : Jay Patel
Roll No : 24BCP291

Lab Experiment 7

Objective:

Implementing the Floyd-Warshall Algorithm for all pair shortest path problem.

Code:

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    long long n;
    cin >> n;
    const long long INF = 1e18;
    vector<vector<long long>> a(n, vector<long long>(n));
    for(long long i = 0; i < n; i++)
    {
        for(long long j = 0; j < n; j++)
        {
            cin >> a[i][j];
            if(a[i][j] == 1000000000)
                a[i][j] = INF;
        }
    }

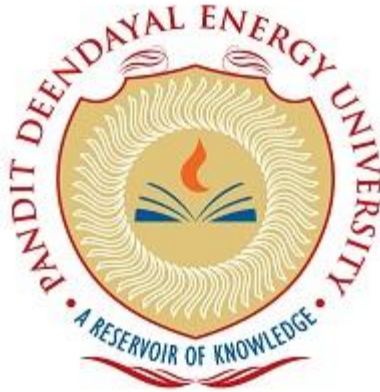
    // Floyd-Warshall Algorithm
    for(long long k = 0; k < n; k++)
    {
        for(long long i = 0; i < n; i++)
        {
            for(long long j = 0; j < n; j++)
            {
                if(a[i][k] != INF && a[k][j] != INF)
                {
                    a[i][j] = min(a[i][j], a[i][k] + a[k][j]);
                }
            }
        }
    }

    cout << "Shortest Path Matrix:\n";
    for(long long i = 0; i < n; i++)
    {
        for(long long j = 0; j < n; j++)
        {
            if(a[i][j] == INF)
                cout << "INF ";
            else
                cout << a[i][j] << " ";
        }
    }
}
```

```
    }  
    cout << endl;  
}  
return 0;  
}
```

OUTPUT:

```
4  
0 5 1000000000 10  
1000000000 0 3 1000000000  
1000000000 1000000000 0 1  
1000000000 1000000000 1000000000 0  
Shortest Path Matrix:  
0 5 8 9  
INF 0 3 4  
INF INF 0 1  
INF INF INF 0
```



Pandit Deendayal Energy University (PDEU)
Department of Computer Science and Engineering,
School of Technology,
Design and Analysis of Algorithm
(24CS210P)

Name : Jay Patel
Roll No : 24BCP291

Lab Experiment 8

Objective:

Solving the **n** queens problem using **backtracking**.

Code:

```
#include <bits/stdc++.h>
using namespace std;

int n;
vector<vector<int>> board;

bool safe(int r, int c)
{
    for(int i = 0; i < r; i++)
        if(board[i][c]) return false;

    for(int i = r - 1, j = c - 1; i >= 0 && j >= 0; i--, j--)
        if(board[i][j]) return false;

    for(int i = r - 1, j = c + 1; i >= 0 && j < n; i--, j++)
        if(board[i][j]) return false;

    return true;
}

bool place(int r)
{
    if(r == n) return true;

    for(int c = 0; c < n; c++)
    {
        if(safe(r, c))
        {
            board[r][c] = 1;
            if(place(r + 1)) return true;
            board[r][c] = 0;
        }
    }
    return false;
}

void printBoard()
{
    cout << "\nChessboard:\n\n";
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
            cout << (board[i][j] ? " Q " : " . ");
```

```

        cout << endl;
    }
}
int main()
{
    cout << "==== N-Queens Problem (Backtracking) =====\n";
    cout << "Enter number of queens (n): ";
    cin >> n;

    board.assign(n, vector<int>(n, 0));

    if(place(0))
    {
        cout << "\nSolution found!\n";
        printBoard();
    }
    else
    {
        cout << "\nNo solution exists for n = " << n << endl;
    }

    return 0;
}

```

OUTPUT:

```

==== N-Queens Problem (Backtracking) =====
Enter number of queens (n): 8

```

Solution found!

Chessboard:

```

Q   .   .   .   .   .   .   .
.   .   .   .   Q   .   .   .
.   .   .   .   .   .   .   Q
.   .   .   .   .   Q   .   .
.   .   Q   .   .   .   .   .
.   .   .   .   .   .   Q   .
.   Q   .   .   .   .   .   .
.   .   .   Q   .   .   .   .

```



Pandit Deendayal Energy University (PDEU)
Department of Computer Science and Engineering,
School of Technology,
Design and Analysis of Algorithm
(24CS210P)

Name : Jay Patel
Roll No : 24BCP291

Lab Experiment 9

Objective:

To find the shortest possible tour that visits every city exactly once and returns to the starting point.

Code:

```
#include <bits/stdc++.h>
using namespace std;

int n, bestCost = INT_MAX;
vector<vector<int>>> cost;
vector<int> bestPath;

int getMin(int i) {
    int m = INT_MAX;
    for (int j = 0; j < n; j++)
        if (i != j) m = min(m, cost[i][j]);
    return m;
}

int getSecondMin(int i) {
    int a = INT_MAX, b = INT_MAX;
    for (int j = 0; j < n; j++) {
        if (i == j) continue;
        if (cost[i][j] <= a) {
            b = a;
            a = cost[i][j];
        } else if (cost[i][j] < b) {
            b = cost[i][j];
        }
    }
    return b;
}

void solve(int bound, int currCost, int level, vector<int>& path, vector<bool>& vis) {
    if (level == n) {
        int total = currCost + cost[path[level - 1]][path[0]];
        if (total < bestCost) {
            bestCost = total;
            bestPath = path;
            bestPath.push_back(path[0]);
        }
        return;
    }

    for (int i = 0; i < n; i++) {
        if (!vis[i] && cost[path[level - 1]][i] != 0)
```

```

{
    int tempBound = bound;
    int tempCost = currCost + cost[path[level - 1]][i];

    if (level == 1)
        bound -= (getMin(path[level - 1]) + getMin(i)) / 2;
    else
        bound -= (getSecondMin(path[level - 1]) + getMin(i)) / 2;
    if (tempCost + bound < bestCost) {
        path[level] = i;
        vis[i] = true;
        solve(bound, tempCost, level + 1, path, vis);
    }
    bound = tempBound;
    vis[i] = false;
}
}
}
int main()
{
    cout << "Enter number of cities: ";
    cin >> n;
    cost.resize(n, vector<int>(n));
    cout << "Enter distance matrix:\n";
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> cost[i][j];
    vector<int> path(n);
    vector<bool> vis(n, false);
    int bound = 0;
    for (int i = 0; i < n; i++)
        bound += getMin(i) + getSecondMin(i);
    bound = (bound + 1) / 2;
    vis[0] = true;
    path[0] = 0;
    solve(bound, 0, 1, path, vis);
    cout << "\nMinimum Cost: " << bestCost << endl;
    cout << "Path: ";
    for (int x : bestPath) cout << x << " ";
    return 0;
}

```


OUTPUT:

```
Enter number of cities: 4
```

```
Enter distance matrix:
```

```
0 10 15 20
```

```
10 0 35 25
```

```
15 35 0 30
```

```
20 25 30 0
```

```
Minimum Cost: 80
```

```
Path: 0 1 3 2 0
```



Pandit Deendayal Energy University (PDEU)
Department of Computer Science and Engineering,
School of Technology,
Design and Analysis of Algorithm
(24CS210P)

Name : Jay Patel

Roll No : 24BCP291

Lab Experiment 10

Objective:

To design and solve given problems using different algorithmic approaches and analyse their complexity.

Code 1(Greedy):

```
#include <bits/stdc++.h>
using namespace std;
struct Item {
    int value, weight;
};
bool cmp(Item a, Item b) {
    return (double)a.value/a.weight > (double)b.value/b.weight;
}
double fractionalKnapsack(int W, vector<Item> &items) {
    sort(items.begin(), items.end(), cmp);

    double totalValue = 0.0;

    for (auto &item : items) {
        if (W >= item.weight) {
            W -= item.weight;
            totalValue += item.value;
        } else {
            totalValue += item.value * ((double)W / item.weight);
            break;
        }
    }
    return totalValue;
}

int main() {
    int n, W;
    cout << "Enter number of items: ";
    cin >> n;

    vector<Item> items(n);

    cout << "Enter weights:\n";
    for (int i = 0; i < n; i++) cin >> items[i].weight;
    cout << "Enter values:\n";
    for (int i = 0; i < n; i++) cin >> items[i].value;
    cout << "Enter capacity: ";
    cin >> W;

    cout << "Max value (Fractional): " << fractionalKnapsack(W, items) << endl;
}
```

OUTPUT:

```
Enter number of items: 4
Enter weights:
2
3
4
5
Enter values:
3
4
5
6
Enter capacity: 5
Max value (Fractional): 7
```

Code 2 (Dynamic Programming):

```
#include <bits/stdc++.h>

using namespace std;

int knapsackDP(int W, vector<int>& wt, vector<int>& val, int n) {
    vector<vector<int>>> dp(n+1, vector<int>(W+1, 0));

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (wt[i-1] <= w)
                dp[i][w] = max(dp[i-1][w], val[i-1] + dp[i-1][w - wt[i-1]]);
            else
                dp[i][w] = dp[i-1][w];
        }
    }

    return dp[n][W];
}
```

```

int main() {
    int n, W;
    cin >> n;
    vector<int> wt(n), val(n);
    for (int i = 0; i < n; i++) cin >> wt[i];
    for (int i = 0; i < n; i++) cin >> val[i];
    cin >> W;
    cout << "Max value (DP): " << knapsackDP(W, wt, val, n);
}

```

OUTPUT:

```

4
2 3 4 5
3 4 5 6
5
Max value (DP): 7

```

Code 3 (Back tracking):

```

#include <bits/stdc++.h>
using namespace std;
int maxProfit = 0;
void backtrack(int i, int n, int W, int currWeight, int currProfit,
               vector<int>& wt, vector<int>& val)
{
    if (i == n) {
        maxProfit = max(maxProfit, currProfit);
        return;
    }
    if (currWeight + wt[i] <= W) {

```

```
        backtrack(i + 1, n, W,
                currWeight + wt[i],
                currProfit + val[i],
                wt, val);
    }
```

```
        backtrack(i + 1, n, W,
                currWeight,
                currProfit,
                wt, val);
    }
```

```
int main() {
    int n, W;
    cout << "Enter number of items: ";
    cin >> n;
    vector<int> wt(n), val(n);
    cout << "Enter weights:\n";
    for (int i = 0; i < n; i++) {
        cout << "Weight of item " << i+1 << ": ";
        cin >> wt[i];
    }
    cout << "Enter values:\n";
    for (int i = 0; i < n; i++) {
        cout << "Value of item " << i+1 << ": ";
        cin >> val[i];
    }
    cout << "Enter knapsack capacity: ";
    cin >> W;
```

```

    backtrack(0, n, W, 0, 0, wt, val);

    cout << "\nMaximum Profit (Backtracking): " << maxProfit << endl;

    return 0;
}

```

OUTPUT:

```

Enter number of items: 4
Enter weights:
Weight of item 1: 2
Weight of item 2: 3
Weight of item 3: 4
Weight of item 4: 5
Enter values:
Value of item 1: 3
Value of item 2: 4
Value of item 3: 5
Value of item 4: 6
Enter knapsack capacity: 5

Maximum Profit (Backtracking): 7

```

Code 4 (Branch and Bound):

```

#include <bits/stdc++.h>

using namespace std;

struct Node {
    int level, profit, weight;

    double bound;
};

bool cmp(pair<int,int> a, pair<int,int> b) {
    return (double)a.second/a.first > (double)b.second/b.first;
}

double bound(Node u, int n, int W, vector<pair<int,int>>& items) {

```

```

    if (u.weight >= W) return 0;
    double profitBound = u.profit;
    int j = u.level + 1;
    int totweight = u.weight;
    while (j < n && totweight + items[j].first <= W) {
        totweight += items[j].first;
        profitBound += items[j].second;
        j++;
    }
    if (j < n)
        profitBound += (W - totweight) *
            ((double)items[j].second / items[j].first);
    return profitBound;
}

int knapsackBB(int W, vector<int>& wt, vector<int>& val, int n) {
    vector<pair<int,int>> items;
    for (int i = 0; i < n; i++)
        items.push_back({wt[i], val[i]});
    sort(items.begin(), items.end(), cmp);
    queue<Node> Q;
    Node u, v;
    v.level = -1;
    v.profit = 0;
    v.weight = 0;
    v.bound = bound(v, n, W, items);
    Q.push(v);
    int maxProfit = 0;
    while (!Q.empty()) {
        v = Q.front();

```



```

        Q.pop();
        if (v.level == n - 1)
            continue;
        u.level = v.level + 1;
        // Include item
        u.weight = v.weight + items[u.level].first;
        u.profit = v.profit + items[u.level].second;
        if (u.weight <= W && u.profit > maxProfit)
            maxProfit = u.profit;
        u.bound = bound(u, n, W, items);
        if (u.bound > maxProfit)
            Q.push(u);
        u.weight = v.weight;
        u.profit = v.profit;
        u.bound = bound(u, n, W, items);
        if (u.bound > maxProfit)
            Q.push(u);
    }
    return maxProfit;
}

int main() {
    int n, W;
    cout << "Enter number of items: ";
    cin >> n;
    vector<int> wt(n), val(n);
    cout << "Enter weights:\n";
    for (int i = 0; i < n; i++) {
        cout << "Weight of item " << i+1 << ": ";
        cin >> wt[i];
    }
}

```

```

    }

    cout << "Enter values:\n";
    for (int i = 0; i < n; i++) {
        cout << "Value of item " << i+1 << ": ";
        cin >> val[i];
    }

    cout << "Enter knapsack capacity: ";
    cin >> W

    int result = knapsackBB(W, wt, val, n);

    cout << "\nMaximum Profit (Branch & Bound): " << result << endl;

    return 0;
}

```

OUTPUT:

```

Enter number of items: 4
Enter weights:
Weight of item 1: 2
Weight of item 2: 3
Weight of item 3: 4
Weight of item 4: 5
Enter values:
Value of item 1: 3
Value of item 2: 4
Value of item 3: 5
Value of item 4: 6
Enter knapsack capacity: 5

Maximum Profit (Branch & Bound): 7

```